Project Executable Files

Date	2 june 2026
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	3 Marks

Project Executable Files

This section covers all executable and deployable files for the OptiCrop project. All files are properly organized, named, and accessible. The evaluator can run or deploy the solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA – CSV dataset used; no relational DB
5	Environment configuration file (.env.example or similar)	Yes – .env.example provided
6	Deployed application URL (if hosted)	Yes – opticrop.onrender.com
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA – Web application
8	Dockerfile / Containerization config (if applicable)	NA – Deployed on Render
9	Test files and test results	Yes – test_model.py included
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Overview of the OptiCrop project file and folder structure:

```
opticrop/
■■■ app.py # Main Flask application (routes, ML inference)
■■■ model.py # ML model training & prediction logic
■■■ train_model.py # Script to train & save all ML models
■■■ requirements.txt # Python dependencies
```

```
■■■ .env.example # Environment variable template
■■■ .gitignore
■■■ README.md # Setup and run instructions
■■■ Crop_recommendation.csv # Training dataset (N, P, K, temp, humidity, pH,
rainfall, label)
■■■ models/ # Saved trained model files
■ ■■■ logistic_regression.pkl
■ ■■■ knn.pkl
■ ■■■ decision_tree.pkl
■ ■■■ random_forest.pkl
■ ■■■ kmeans.pkl
■■■ static/
■ ■■■ css/style.css # Responsive dark-theme stylesheet
■ ■■■ js/main.js # Frontend interaction scripts
■■■ templates/
■ ■■■ index.html # Home page (hero, CTA buttons)
■ ■■■ analyze.html # Crop Feasibility Analyzer form
■ ■■■ result.html # Results / recommendation display
■■■ tests/
■■■ test_model.py # Unit tests for ML prediction pipeline
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://opticrop.onrender.com
Login Credentials (Demo)	No login required – the crop analysis tool is publicly accessible without authentication
Platform / Hosting Provider	Render (render.com) – free tier web service hosting the Flask application
Repository Link	https://github.com/opticrop-team/opticrop
Demo Video Link	Available in submission portal / shared via team drive

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project locally:

Pre-requisites:
Python 3.8+, pip, Git. Recommended: VS Code or any Python IDE.

Step 1 – Clone the repository:

```
git clone https://github.com/opticrop-team/opticrop.git && cd opticrop
```

Step 2 – Create and activate a virtual environment:

```
python -m venv venv venv\Scripts\activate (Windows) source venv/bin/activate (Mac/Linux)
```

Step 3 – Install dependencies:

```
pip install -r requirements.txt
```

Step 4 – Configure environment (optional for hosted version):

```
Copy .env.example to .env — no API keys required; the ML models run locally.
```

Step 5 – Train models (first-time only, pre-trained models included):

```
python train_model.py # generates .pkl files in /models directory
```

Step 6 – Start the Flask application:

```
python app.py
```

Step 7 – Open browser:

```
Navigate to http://127.0.0.1:5000 or http://localhost:5000
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Cold start latency on Render free tier: first request after server inactivity takes 8–12 seconds as Render spins up the instance	Resolved by visiting the live URL once before demo; subsequent requests are fast (< 2 sec)
2	The ML model accepts only the 7 defined input features (N, P, K, temp, humidity, pH, rainfall); inputs outside expected ranges may produce lower-confidence predictions	Input validation on the frontend enforces non-negative numerical values and guides users with placeholder ranges
3	The Render free tier has limited RAM (~512 MB); loading all 5 ML model pickle files simultaneously may approach memory limits under heavy concurrent load	Models are loaded once at startup and cached in memory; low-memory models (Scikit-learn) are used to minimize footprint; upgrade to paid tier for production scale